# Terraformで素数をマイニングしたら、ブロックチェーンが生まれた

～ Infrastructure as Code。つまりCode。なら何でもできますよね？ ～

田中博悠 / tanahiro2010

# 田中博悠 / tanahiro2010

- 三田学園高等学校 1年
- TypeScript / Web / CLI / AI Agent / Security
- 個人開発・OSS・脆弱性調査・イベント運営

# Terraform、使ってますか？

# インフラをコードで管理するやつ

# Infrastructure as Code

# Infrastructure as Code

# つまり

# Code

# Code なら

素数のマイニングもできますよね？



できました

# 今日の話

1. **Prime Discovery** — HCL だけで素数を探す
2. **Recursive Terraform** — Terraform が Terraform を起動する
3. **Distributed Prime Mining** — Terraform を並列で動かす
4. **PrimeChain** — 素数でブロックチェーンを作る
5. **Terraform Destroy** — 全部壊す

# ルール

- 計算ロジックは **すべて HCL**
- Python / JS / Go などで計算するのは禁止
- シェルは「Terraform の起動」と「ファイル配置」だけ
- Provider は Terraform 組み込みのものだけ
  - `terraform_data` / `terraform_remote_state`

# Terraform 以外に計算させたら負け

# Phase 1

# Prime Discovery

# HCL にはループがない

でも `for` 式はある

`n` が素数 ⇔ `2..√n` のどれでも割り切れない

```hcl
locals {
  candidates = range(var.start, var.end)

  # n が素数 ⇔ 2..floor(√n) のどれでも割り切れない
  primes = [
    for n in local.candidates : n
    if alltrue([for d in range(2, floor(pow(n, 0.5)) + 1) : n % d != 0])
  ]
}
```

`terraform apply` すると

```
$ terraform apply -var start=1 -var end=100

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
          43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
summary = "[1, 100) の 98 個の整数を判定し、25 個の素数を発見しました（最大: 97）"
```

**Resources: 0 added**

# インフラは何も作っていない

terraform apply が素数を計算しているだけ

# ハマりポイント

`range()` は **1024 要素** までしか
作れない

```
$ echo 'length(range(1025))' | terraform console

Error: Error in function call
```

```hcl
locals {
  # range() は 1 回で最大 1024 要素まで
  # → 探索範囲を 1024 ずつのチャンクに分割してから flatten する
  chunk_starts = range(local.lower, var.end, 1024)
  candidates = flatten([
    for s in local.chunk_starts : range(s, min(s + 1024, var.end))
  ])
}
```

除数側の range(2, √n) も 1024 以内 → n は $1024^2$ = 1,048,576 まで

# 10 万までやってみた

```
$ make prime START=1 END=100000

summary = "[1, 100000) の 99998 個の整数を判定し、9592 個の素数を発見しました（最大: 99991）"

real  2:02.14
```

2 分

遅い

# Phase 2

# Recursive Terraform

# 範囲を分割したい

# でも HCL には再帰関数もない

なら

# Terraform を再帰起動すればいい

# 自分自身を子 Terraform として起動する

```hcl
module "child" {
  source   = "../modules/spawn"
  for_each = local.is_leaf ? {} : {
    L = { start = var.start, end = local.mid }
    R = { start = local.mid, end = var.end }
  }

  name       = "${var.node_id}.${each.key}"
  config_dir = abspath(path.root) # ← 自分自身
  vars = {
    start = tostring(each.value.start)
    end   = tostring(each.value.end)
  }
}
```

# spawn モジュール

```hcl
resource "terraform_data" "process" {
  triggers_replace = var.vars
  input            = { env = local.env } # TF_VAR_* と TF_DATA_DIR

  provisioner "local-exec" {
    environment = self.input.env
    command     = <<-EOT
      terraform -chdir="$CONFIG_DIR" init \
        -backend-config="path=$RUN_DIR/terraform.tfstate"
      terraform -chdir="$CONFIG_DIR" apply -auto-approve
    EOT
  }
}
```

# 子の結果は子の State から読む

```hcl
data "terraform_remote_state" "process" {
  backend = "local"
  config = {
    path = "${local.run_dir}/terraform.tfstate"
  }

  depends_on = [terraform_data.process] # 子の apply が終わってから読む
}
```

引数は TF_VAR_*、戻り値は terraform_remote_state。プロセス間通信です

```
$ make recursive END=8000 LEAF=1000

🚀 23:33:14 spawn terraform: root.L
🚀 23:33:14 spawn terraform: root.R
✅ 23:33:15 done:            root.L
✅ 23:33:15 done:            root.R

tree = [
  "root [1, 8000) → split",
  "    root.L [1, 4000) → split",
  "        root.L.L [1, 2000) → split",
  "            root.L.L.L [1, 1000) → 168 primes",
  "            root.L.L.R [1000, 2000) → 135 primes",
  "        root.L.R [2000, 4000) → split",
  ...
  "            root.R.R.R [7000, 8000) → 107 primes",
]
```

15 個の Terraform が

自分自身を起動している

# ハマりポイント

１回目は動くのに

２回目の apply で子が全滅

```
$ terraform apply -var end=3000

🚀 23:31:35 spawn terraform: root.L
Error: Error acquiring the state lock
Error message: resource temporarily unavailable
Lock Info:
  Path:      terraform.tfstate
  Operation: OperationTypeApply
```

子の `terraform init` が

親の `terraform.tfstate` を見つけて

「**古い State を移行しなきゃ！**」

→ 親が掴んでいるロックとぶつかる

```hcl
terraform {
  # 直接 apply したときの State も config_dir 直下に置かない
  # (./terraform.tfstate があると子の init が「移行」しに行く)
  backend "local" {
    path = ".runs/root/terraform.tfstate"
  }
}
```

# Phase 3

# Distributed Prime Mining

再帰できるなら

並列もできる

# Pool と Miner

```hcl
locals {
  assignments = {
    for i in range(var.miners) : format("miner-%02d", i + 1) => {
      start = var.start + i * local.span
      end   = min(var.start + (i + 1) * local.span, var.end)
    }
  }
}

module "miner" {
  source   = "../modules/spawn"
  for_each = local.assignments
  name     = each.key
}
```

Miner ごとに State を分けるので、ロック競合は起きない

```
$ make pool

🚀 23:33:44 spawn terraform: miner-04
🚀 23:33:44 spawn terraform: miner-01
🚀 23:33:44 spawn terraform: miner-03
🚀 23:33:44 spawn terraform: miner-02
✅ 23:33:46 done:            miner-01
✅ 23:33:47 done:            miner-02
✅ 23:33:48 done:            miner-03
✅ 23:33:48 done:            miner-04

summary = "4 台の Terraform Miner が [1, 20000) から 2262 個の素数を発見しました（最大: 19997）"
```

Terraform はリソースを 10 並列 で作る

＝ Miner が勝手に並列で動く

# [1, 100000) の素数 9592 個

| 構成 | 時間 |
| --- | --- |
| Phase 1: 1 プロセス | 2 分 02 秒 |
| Phase 3: 8 Miner | **38 秒** |

# 3.2 倍

# （Python なら一瞬）

言わない約束

# Phase 4

# PrimeChain

# 素数が掘れるようになった

# 掘る（マイニング）といえば

# ブロックチェーン

# ブロック

| フィールド | 中身 |
| --- | --- |
| `height` | ブロック番号 |
| `prev_hash` | 前ブロックのハッシュ |
| `miner` | 掘った Miner |
| `prime` | 発見した素数 |
| `nonce` | Nonce |
| `hash` | `sha256("height\|prev_hash\|miner\|prime\|nonce")` |

`terraform apply` 1回＝**ブロック1個**

# 採掘も HCL

```hcl
locals {
  # 見つけた素数 × nonce の全組み合わせで sha256 を計算
  attempts = flatten([
    for p in module.prime.primes : [
      for n in range(var.max_nonce) : {
        prime = p
        nonce = n
        hash  = sha256(format("%d|%s|%s|%d|%d",
          var.height, var.prev_hash, var.miner_id, p, n))
      }
    ]
  ])

  # 先頭が 0 * difficulty のものだけ残す
  hits = [for a in local.attempts : a if startswith(a.hash, local.target)]
}
```

採用ルール：

ハッシュ値が いちばん小さい 候補

```
$ make mine BLOCKS=3

🚀 23:37:49 spawn terraform: miner-04
🚀 23:37:49 spawn terraform: miner-01
...
⛏️ Block #0 mined by miner-01: prime=101 nonce=23 hash=001817dc39715b1f... (4 candidates, 5056 hashes)
⛏️ Block #1 mined by miner-02: prime=593 nonce=19 hash=0008d290675e22b3... (4 candidates, 3840 hashes)
⛏️ Block #2 mined by miner-04: prime=1103 nonce=48 hash=001d824ab4d1be8c... (4 candidates, 3712 hashes)

$ make explorer

#0   001817dc39715b1f ← 0000000000000000  miner-01 prime=101    nonce=23
#1   0008d290675e22b3 ← 001817dc39715b1f  miner-02 prime=593    nonce=19
#2   001d824ab4d1be8c ← 0008d290675e22b3  miner-04 prime=1103   nonce=48
```

# チェーンの検証も HCL

```hcl
verifications = [
  for i, b in local.chain : {
    height = b.height == i
    link   = b.prev_hash == try(local.chain[i - 1].hash, local.genesis_hash)
    hash   = b.hash == sha256(format("%d|%s|%s|%d|%d",
               b.height, b.prev_hash, b.miner, b.prime, b.nonce))
    work   = startswith(b.hash, join("", [for _ in range(b.difficulty) : "0"]))
    prime  = b.prime >= 2 && alltrue([
      for d in range(2, max(2, floor(pow(b.prime, 0.5)) + 1)) : b.prime % d != 0
    ])
  }
]
```

# 1 ブロック = 1 リソース

```hcl
resource "terraform_data" "block" {
  for_each = { for i in range(local.height + 1) : format("%04d", i) => i }
  input    = local.chain[each.value]

  lifecycle {
    precondition {
      condition     = alltrue(values(local.verifications[each.value]))
      error_message = "ブロック #${each.value} の検証に失敗しました"
    }
  }
}
```

# 改ざんしてみる

```
$ make tamper BLOCK=1
😈 Block #1 の prime を 4 に書き換えました

$ make mine

Error: Resource precondition failed

  on main.tf line 173, in resource "terraform_data" "block":
 173:       condition     = try(alltrue(values(local.verifications[each.value])), false)

ブロック #1 の検証に失敗しました:
{"hash":false,"height":true,"link":true,"prime":false,"work":true}
make: *** [mine] Error 1
```

# plan の時点でバレる

Miner を起動する前に止まる

# ハマりポイント

# 台帳をどこに置くか

# 最初の案：

# 自分自身の `terraform.tfstate` を `jsondecode(file())` で読む

```
$ terraform apply

Error: Error in function call

Call to function "try" failed: panic in function implementation:
returned value cty.EmptyTupleVal does not conform to expected return type
...
goroutine 397 [running]:
runtime/debug.Stack()
```

# panic

# apply 中に Terraform が State を書き換える

→ plan と apply で読める値が変わる

# 検証済みのチェーンだけを

# ledger.json に書き出すことにした

中身は HCL が jsonencode した JSON。シェルはファイルに置くだけ

# Phase 5

# Terraform Destroy

# 最後に

# Terraform 本来の機能を使います

```
terraform destroy
```

# 親が消えたら子も消す

```hcl
provisioner "local-exec" {
  when        = destroy
  environment = self.input.env
  command     = <<-EOT
    terraform -chdir="$CONFIG_DIR" destroy -auto-approve
    rm -rf "$RUN_DIR"
    echo "💥 destroyed: $RUN_NAME"
  EOT
}
```

子が孫を持っていれば、孫も連鎖して destroy される

```
$ make destroy

== terraform destroy: chain
💥 ledger.json destroyed. PrimeChain is gone.
💥 Block #0 destroyed. prime 101 is gone.
💥 Block #2 destroyed. prime 1103 is gone.
💥 Block #1 destroyed. prime 593 is gone.
💥 23:39:16 destroyed:       miner-02
💥 23:39:16 destroyed:       miner-04
💥 23:39:16 destroyed:       miner-01
💥 23:39:16 destroyed:       miner-03
Destroy complete! Resources: 9 destroyed.
```

```
== terraform destroy: recursive
💥 23:39:19 destroyed:       root.R.L.L
💥 23:39:19 destroyed:       root.R.L.R
💥 23:39:19 destroyed:       root.R.L
💥 23:39:19 destroyed:       root.R.R.L
💥 23:39:19 destroyed:       root.R.R.R
💥 23:39:19 destroyed:       root.R.R
💥 23:39:19 destroyed:       root.L.L.L
💥 23:39:19 destroyed:       root.L.L.R
💥 23:39:19 destroyed:       root.L.L
💥 23:39:19 destroyed:       root.L.R.R
💥 23:39:19 destroyed:       root.L.R.L
💥 23:39:19 destroyed:       root.L.R
💥 23:39:19 destroyed:       root.R
💥 23:39:19 destroyed:       root.L
Destroy complete! Resources: 2 destroyed.
```

親から見ると リソース２個

その裏で 14 個の Terraform が
消えていく

# そして Phase 1 の素数は

```
== terraform destroy: prime-discovery
Destroy complete! Resources: 0 destroyed.
```

そもそもリソースじゃなかった

# Terraform で素数を構築し、
# Terraform で素数を破壊する

# まとめ

# 分かったこと

- HCL は思ったより書ける
  - `for` 式・`alltrue`・`sha256`・`precondition`
- `local-exec` + `TF_VAR_*` + `terraform_remote_state` で **プロセス間通信** ができる
- Terraform は **State を中心に** 動いている
  - init は State の移行を気にするし、apply 中は State を書き換え続ける
- `range()` の 1024 制限など、計算基盤ではない前提が随所にある

技術的に可能なことと

技術的に適切なことは違う

しかし、それを知るためには
時として技術を間違った方向に
**全力で使う** 必要がある

# All You Need Is Terraform.

# ありがとうございました

Terraform PrimeChain

田中博悠 / tanahiro2010